

My Project

Generated by Doxygen 1.13.2

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 AndGate Class Reference	7
4.1.1 Member Function Documentation	8
4.1.1.1 evaluate()	8
4.1.1.2 print()	8
4.2 Circuit Class Reference	8
4.3 Component Class Reference	9
4.4 Gate Class Reference	10
4.4.1 Member Function Documentation	11
4.4.1.1 print()	11
4.5 LogicVector Class Reference	12
4.6 NandGate Class Reference	12
4.6.1 Member Function Documentation	13
4.6.1.1 evaluate()	13
4.6.1.2 print()	13
4.7 NorGate Class Reference	14
4.7.1 Member Function Documentation	15
4.7.1.1 evaluate()	15
4.7.1.2 print()	15
4.8 NotGate Class Reference	15
4.8.1 Member Function Documentation	16
4.8.1.1 evaluate()	16
4.8.1.2 print()	17
4.9 OrGate Class Reference	17
4.9.1 Member Function Documentation	18
4.9.1.1 evaluate()	18
4.9.1.2 print()	18
4.10 Pin Class Reference	18
4.10.1 Member Function Documentation	19
4.10.1.1 evaluate()	19
4.10.1.2 print()	20
4.10.1.3 reset()	20
4.11 Port Class Reference	20
4.12 Project Class Reference	21

4.13 Simulator Class Reference	22
4.14 TestProject Class Reference	22
4.15 Wire Class Reference	23
4.16 XnorGate Class Reference	24
4.16.1 Member Function Documentation	25
4.16.1.1 evaluate()	25
4.16.1.2 print()	25
4.17 XorGate Class Reference	25
4.17.1 Member Function Documentation	26
4.17.1.1 evaluate()	26
4.17.1.2 print()	26
5 File Documentation	27
5.1 Adder.h	27
5.2 Circuit.h	27
5.3 Clock.h	31
5.4 Component.h	31
5.5 Gate.h	33
5.6 Gates.h	34
5.7 LogicValue.h	36
5.8 LogicVector.h	38
5.9 Mux.h	39
5.10 Pin.h	39
5.11 Port.h	41
5.12 RAM.h	42
5.13 Register.h	43
5.14 ROM.h	43
5.15 Simulator.h	43
5.16 Test.h	45
5.17 Wire.h	47
Index	49

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Circuit	8
Project	21
Component	9
Gate	10
AndGate	7
NandGate	12
NorGate	14
NotGate	15
OrGate	17
XnorGate	24
XorGate	25
Pin	18
LogicVector	12
Port	20
Simulator	22
TestProject	22
Wire	23

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AndGate	7
Circuit	8
Component	9
Gate	10
LogicVector	12
NandGate	12
NorGate	14
NotGate	15
OrGate	17
Pin	18
Port	20
Project	21
Simulator	22
TestProject	22
Wire	23
XnorGate	24
XorGate	25

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

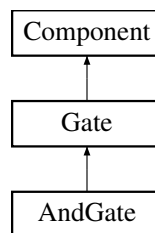
/home/student/Desktop/Project/Logisimp/include/Adder.h	27
/home/student/Desktop/Project/Logisimp/include/Circuit.h	27
/home/student/Desktop/Project/Logisimp/include/Clock.h	31
/home/student/Desktop/Project/Logisimp/include/Component.h	31
/home/student/Desktop/Project/Logisimp/include/Gate.h	33
/home/student/Desktop/Project/Logisimp/include/Gates.h	34
/home/student/Desktop/Project/Logisimp/include/LogicValue.h	36
/home/student/Desktop/Project/Logisimp/include/LogicVector.h	38
/home/student/Desktop/Project/Logisimp/include/Mux.h	39
/home/student/Desktop/Project/Logisimp/include/Pin.h	39
/home/student/Desktop/Project/Logisimp/include/Port.h	41
/home/student/Desktop/Project/Logisimp/include/RAM.h	42
/home/student/Desktop/Project/Logisimp/include/Register.h	43
/home/student/Desktop/Project/Logisimp/include/ROM.h	43
/home/student/Desktop/Project/Logisimp/include/Simulator.h	43
/home/student/Desktop/Project/Logisimp/include/Test.h	45
/home/student/Desktop/Project/Logisimp/include/Wire.h	47

Chapter 4

Class Documentation

4.1 AndGate Class Reference

Inheritance diagram for AndGate:



Public Member Functions

- **AndGate** (int id, int input_count=2)
- void **evaluate** ([Simulator](#) &simulator)
- void **print** (std::ostream &cout) const

Public Member Functions inherited from [Gate](#)

- **Gate** (int id, int input_count)
- [Port](#) * **output** ()
- [Port](#) * **output** () const
- void **print** (std::ostream &cout) const

Public Member Functions inherited from [Component](#)

- **Component** (int id, const std::string &label="")
- **Component** (const [Component](#) &other)=delete
- [Component](#) & **operator=** (const [Component](#) &other)=delete
- **Component** ([Component](#) &&other)=delete
- [Component](#) & **operator=** ([Component](#) &&other)=delete
- int **id** () const
- std::string & **label** ()

- `std::string label () const`
- `std::vector< Port * > & inputs ()`
- `std::vector< Port * > inputs () const`
- `std::vector< Port * > & outputs ()`
- `std::vector< Port * > outputs () const`
- `virtual void reset ()`
- `Port * add_input (int width=1, const std::string &label="")`
- `Port * add_output (int width=1, const std::string &label="")`
- `Port * input_port (int i)`
- `Port * input_port (int i) const`
- `Port * output_port (int i)`
- `Port * output_port (int i) const`
- `int input_count () const`
- `int output_count () const`

Additional Inherited Members

Protected Member Functions inherited from [Gate](#)

- `LogicValue input_value (int index) const`
- `void drive_output (LogicValue value, Simulator &simulator)`

4.1.1 Member Function Documentation

4.1.1.1 evaluate()

```
void AndGate::evaluate (
    Simulator & simulator) [inline], [virtual]
```

Implements [Component](#).

4.1.1.2 print()

```
void AndGate::print (
    std::ostream & cout) const [inline], [virtual]
```

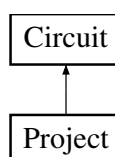
Reimplemented from [Component](#).

The documentation for this class was generated from the following file:

- `/home/student/Desktop/Project/Logisimp/include/Gates.h`

4.2 Circuit Class Reference

Inheritance diagram for Circuit:



Public Member Functions

- int **comp_size** () const
- [Component](#) *& **component_at** (int i)
- [Component](#) * **component_at** (int i) const
- [Circuit](#) (const [Circuit](#) &other)=delete
- [Circuit](#) & **operator=** (const [Circuit](#) &other)=delete
- std::vector< [Component](#) * > & **components** ()
- std::vector< [Component](#) * > **components** () const
- std::vector< [Wire](#) * > & **wires** ()
- std::vector< [Wire](#) * > **wires** () const
- [AndGate](#) & **create_and_gate** (int input_count=2)
- [OrGate](#) & **create_or_gate** (int input_count=2)
- [NotGate](#) & **create_not_gate** ()
- [NandGate](#) & **create_nand_gate** (int input_count=2)
- [NorGate](#) & **create_nor_gate** (int input_count=2)
- [XorGate](#) & **create_xor_gate** (int input_count=2)
- [XnorGate](#) & **create_xnor_gate** (int input_count=2)
- [Pin](#) & **create_pin** (PinType type, int width=1)
- [Wire](#) & **create_wire** (int width=1, const std::string &label="")
- void **connect** ([Port](#) &port, [Wire](#) &wire)
- void **disconnect** ([Port](#) &port)
- void **remove_component** ([Component](#) &component)
- void **remove_wire** ([Wire](#) &wire)
- bool **initialize** ([Simulator](#) &simulator)
- bool **reset** ([Simulator](#) &simulator)

Private Member Functions

- void **store_component** ([Component](#) *component)
- bool **owns_component** (const [Component](#) &component) const
- bool **owns_wire** (const [Wire](#) &wire) const

Private Attributes

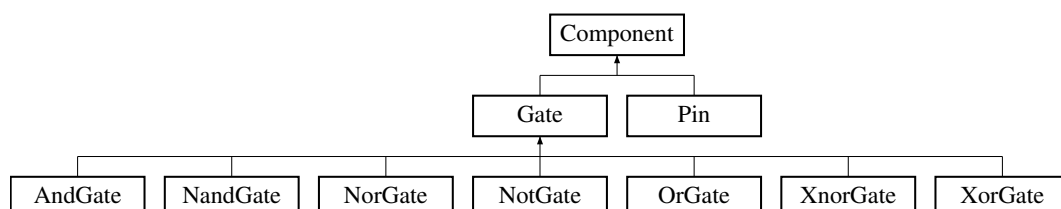
- std::vector< [Component](#) * > **components_**
- std::vector< [Wire](#) * > **wires_**
- int **next_component_id_**
- int **next_wire_id_**

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Logisimp/include/Circuit.h

4.3 Component Class Reference

Inheritance diagram for Component:



Public Member Functions

- **Component** (int id, const std::string &label="")
- **Component** (const [Component](#) &other)=delete
- **Component** & **operator=** (const [Component](#) &other)=delete
- **Component** ([Component](#) &&other)=delete
- **Component** & **operator=** ([Component](#) &&other)=delete
- int **id** () const
- std::string & **label** ()
- std::string **label** () const
- std::vector< [Port](#) * > & **inputs** ()
- std::vector< [Port](#) * > **inputs** () const
- std::vector< [Port](#) * > & **outputs** ()
- std::vector< [Port](#) * > **outputs** () const
- virtual void **evaluate** ([Simulator](#) &simulator)=0
- virtual void **reset** ()
- virtual void **print** (std::ostream &cout) const
- [Port](#) * **add_input** (int width=1, const std::string &label="")
- [Port](#) * **add_output** (int width=1, const std::string &label="")
- [Port](#) * **input_port** (int i)
- [Port](#) * **input_port** (int i) const
- [Port](#) * **output_port** (int i)
- [Port](#) * **output_port** (int i) const
- int **input_count** () const
- int **output_count** () const

Private Member Functions

- void **clear_ports** ()

Private Attributes

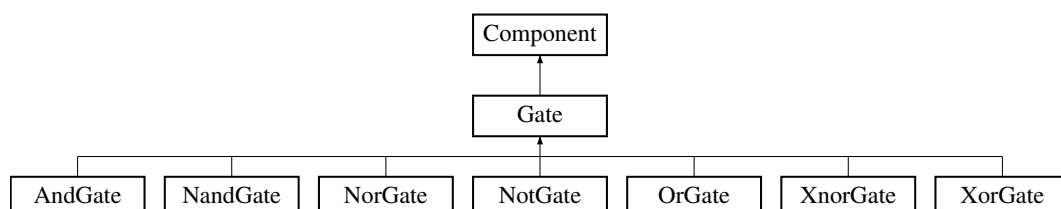
- int **id_**
- std::string **label_**
- std::vector< [Port](#) * > **inputs_**
- std::vector< [Port](#) * > **outputs_**

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Logisimp/include/Component.h

4.4 Gate Class Reference

Inheritance diagram for Gate:



Public Member Functions

- **Gate** (int id, int input_count)
- **Port * output** ()
- **Port * output** () const
- void **print** (std::ostream &cout) const

Public Member Functions inherited from **Component**

- **Component** (int id, const std::string &label="")
- **Component** (const **Component** &other)=delete
- **Component** & **operator=** (const **Component** &other)=delete
- **Component** (**Component** &&other)=delete
- **Component** & **operator=** (**Component** &&other)=delete
- int **id** () const
- std::string & **label** ()
- std::string **label** () const
- std::vector< **Port** * > & **inputs** ()
- std::vector< **Port** * > **inputs** () const
- std::vector< **Port** * > & **outputs** ()
- std::vector< **Port** * > **outputs** () const
- virtual void **evaluate** (**Simulator** &simulator)=0
- virtual void **reset** ()
- **Port** * **add_input** (int width=1, const std::string &label="")
- **Port** * **add_output** (int width=1, const std::string &label="")
- **Port** * **input_port** (int i)
- **Port** * **input_port** (int i) const
- **Port** * **output_port** (int i)
- **Port** * **output_port** (int i) const
- int **input_count** () const
- int **output_count** () const

Protected Member Functions

- LogicValue **input_value** (int index) const
- void **drive_output** (LogicValue value, **Simulator** &simulator)

4.4.1 Member Function Documentation

4.4.1.1 print()

```
void Gate::print (
    std::ostream & cout) const [inline], [virtual]
```

Reimplemented from **Component**.

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Logisimp/include/Gate.h

4.5 LogicVector Class Reference

Public Member Functions

- **LogicVector** (int width=1, LogicValue initial_value=UNKNOWN)
- std::vector< LogicValue > & **bits** ()
- std::vector< LogicValue > **bits** () const
- int **width** () const
- LogicValue **operator[]** (int index) const
- LogicValue & **operator[]** (int index)
- bool **operator==** (const [LogicVector](#) &other) const
- bool **operator!=** (const [LogicVector](#) &other) const
- bool **fully_known** () const

Private Attributes

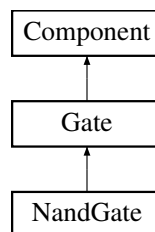
- std::vector< LogicValue > **bits_**

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Logisimp/include/LogicVector.h

4.6 NandGate Class Reference

Inheritance diagram for NandGate:



Public Member Functions

- **NandGate** (int id, int input_count=2)
- void **evaluate** ([Simulator](#) &simulator)
- void **print** (std::ostream &cout) const

Public Member Functions inherited from [Gate](#)

- **Gate** (int id, int input_count)
- [Port](#) * **output** ()
- [Port](#) * **output** () const

Public Member Functions inherited from [Component](#)

- **Component** (int id, const std::string &label="")
- **Component** (const [Component](#) &other)=delete
- **Component** & **operator=** (const [Component](#) &other)=delete
- **Component** ([Component](#) &&other)=delete
- **Component** & **operator=** ([Component](#) &&other)=delete
- int **id** () const
- std::string & **label** ()
- std::string **label** () const
- std::vector< [Port](#) * > & **inputs** ()
- std::vector< [Port](#) * > **inputs** () const
- std::vector< [Port](#) * > & **outputs** ()
- std::vector< [Port](#) * > **outputs** () const
- virtual void **reset** ()
- [Port](#) * **add_input** (int width=1, const std::string &label="")
- [Port](#) * **add_output** (int width=1, const std::string &label="")
- [Port](#) * **input_port** (int i)
- [Port](#) * **input_port** (int i) const
- [Port](#) * **output_port** (int i)
- [Port](#) * **output_port** (int i) const
- int **input_count** () const
- int **output_count** () const

Additional Inherited Members

Protected Member Functions inherited from [Gate](#)

- LogicValue **input_value** (int index) const
- void **drive_output** (LogicValue value, [Simulator](#) &simulator)

4.6.1 Member Function Documentation

4.6.1.1 evaluate()

```
void NandGate::evaluate (
    Simulator & simulator) [inline], [virtual]
```

Implements [Component](#).

4.6.1.2 print()

```
void NandGate::print (
    std::ostream & cout) const [inline], [virtual]
```

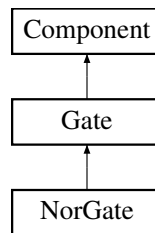
Reimplemented from [Gate](#).

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Logisimp/include/Gates.h

4.7 NorGate Class Reference

Inheritance diagram for NorGate:



Public Member Functions

- **NorGate** (int id, int input_count=2)
- void **evaluate** ([Simulator](#) &simulator)
- void **print** (std::ostream &cout) const

Public Member Functions inherited from [Gate](#)

- **Gate** (int id, int input_count)
- [Port](#) * **output** ()
- [Port](#) * **output** () const

Public Member Functions inherited from [Component](#)

- **Component** (int id, const std::string &label="")
- **Component** (const [Component](#) &other)=delete
- [Component](#) & **operator=** (const [Component](#) &other)=delete
- **Component** ([Component](#) &&other)=delete
- [Component](#) & **operator=** ([Component](#) &&other)=delete
- int **id** () const
- std::string & **label** ()
- std::string **label** () const
- std::vector< [Port](#) * > & **inputs** ()
- std::vector< [Port](#) * > **inputs** () const
- std::vector< [Port](#) * > & **outputs** ()
- std::vector< [Port](#) * > **outputs** () const
- virtual void **reset** ()
- [Port](#) * **add_input** (int width=1, const std::string &label="")
- [Port](#) * **add_output** (int width=1, const std::string &label="")
- [Port](#) * **input_port** (int i)
- [Port](#) * **input_port** (int i) const
- [Port](#) * **output_port** (int i)
- [Port](#) * **output_port** (int i) const
- int **input_count** () const
- int **output_count** () const

Additional Inherited Members

Protected Member Functions inherited from [Gate](#)

- `LogicValue input_value` (int index) const
- `void drive_output` (LogicValue value, [Simulator](#) &simulator)

4.7.1 Member Function Documentation

4.7.1.1 evaluate()

```
void NorGate::evaluate (
    Simulator & simulator) [inline], [virtual]
```

Implements [Component](#).

4.7.1.2 print()

```
void NorGate::print (
    std::ostream & cout) const [inline], [virtual]
```

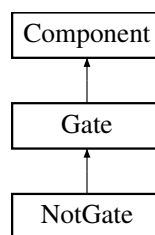
Reimplemented from [Gate](#).

The documentation for this class was generated from the following file:

- `/home/student/Desktop/Project/Logisimp/include/Gates.h`

4.8 NotGate Class Reference

Inheritance diagram for NotGate:



Public Member Functions

- **NotGate** (int id)
- `void evaluate` ([Simulator](#) &simulator)
- `void print` (std::ostream &cout) const

Public Member Functions inherited from [Gate](#)

- **Gate** (int id, int input_count)
- [Port](#) * **output** ()
- [Port](#) * **output** () const

Public Member Functions inherited from [Component](#)

- **Component** (int id, const std::string &label="")
- **Component** (const [Component](#) &other)=delete
- [Component](#) & **operator=** (const [Component](#) &other)=delete
- **Component** ([Component](#) &&other)=delete
- [Component](#) & **operator=** ([Component](#) &&other)=delete
- int **id** () const
- std::string & **label** ()
- std::string **label** () const
- std::vector< [Port](#) * > & **inputs** ()
- std::vector< [Port](#) * > **inputs** () const
- std::vector< [Port](#) * > & **outputs** ()
- std::vector< [Port](#) * > **outputs** () const
- virtual void **reset** ()
- [Port](#) * **add_input** (int width=1, const std::string &label="")
- [Port](#) * **add_output** (int width=1, const std::string &label="")
- [Port](#) * **input_port** (int i)
- [Port](#) * **input_port** (int i) const
- [Port](#) * **output_port** (int i)
- [Port](#) * **output_port** (int i) const
- int **input_count** () const
- int **output_count** () const

Additional Inherited Members

Protected Member Functions inherited from [Gate](#)

- LogicValue **input_value** (int index) const
- void **drive_output** (LogicValue value, [Simulator](#) &simulator)

4.8.1 Member Function Documentation

4.8.1.1 evaluate()

```
void NotGate::evaluate (
    Simulator & simulator) [inline], [virtual]
```

Implements [Component](#).

4.8.1.2 print()

```
void NotGate::print (
    std::ostream & cout) const [inline], [virtual]
```

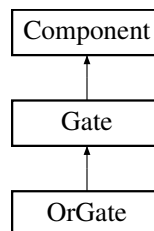
Reimplemented from [Gate](#).

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Logisimp/include/Gates.h

4.9 OrGate Class Reference

Inheritance diagram for OrGate:



Public Member Functions

- **OrGate** (int id, int input_count=2)
- void **evaluate** ([Simulator](#) &simulator)
- void **print** (std::ostream &cout) const

Public Member Functions inherited from [Gate](#)

- **Gate** (int id, int input_count)
- [Port](#) * **output** ()
- [Port](#) * **output** () const

Public Member Functions inherited from [Component](#)

- **Component** (int id, const std::string &label="")
- **Component** (const [Component](#) &other)=delete
- [Component](#) & **operator=** (const [Component](#) &other)=delete
- **Component** ([Component](#) &&other)=delete
- [Component](#) & **operator=** ([Component](#) &&other)=delete
- int **id** () const
- std::string & **label** ()
- std::string **label** () const
- std::vector< [Port](#) * > & **inputs** ()
- std::vector< [Port](#) * > **inputs** () const
- std::vector< [Port](#) * > & **outputs** ()
- std::vector< [Port](#) * > **outputs** () const

- virtual void **reset** ()
- [Port](#) * **add_input** (int width=1, const std::string &label="")
- [Port](#) * **add_output** (int width=1, const std::string &label="")
- [Port](#) * **input_port** (int i)
- [Port](#) * **input_port** (int i) const
- [Port](#) * **output_port** (int i)
- [Port](#) * **output_port** (int i) const
- int **input_count** () const
- int **output_count** () const

Additional Inherited Members

Protected Member Functions inherited from [Gate](#)

- LogicValue **input_value** (int index) const
- void **drive_output** (LogicValue value, [Simulator](#) &simulator)

4.9.1 Member Function Documentation

4.9.1.1 evaluate()

```
void OrGate::evaluate (
    Simulator & simulator) [inline], [virtual]
```

Implements [Component](#).

4.9.1.2 print()

```
void OrGate::print (
    std::ostream & cout) const [inline], [virtual]
```

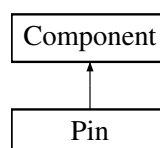
Reimplemented from [Gate](#).

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Logisimp/include/Gates.h

4.10 Pin Class Reference

Inheritance diagram for Pin:



Public Member Functions

- **Pin** (int id, PinType type, int width=1)
- PinType & **type** ()
- PinType **type** () const
- LogicVector & **value** ()
- LogicVector **value** () const
- void **set** (const LogicVector &value, Simulator &simulator)
- void **toggle** (Simulator &simulator)
- void **evaluate** (Simulator &simulator)
- void **reset** ()
- void **print** (std::ostream &cout) const

Public Member Functions inherited from Component

- **Component** (int id, const std::string &label="")
- **Component** (const Component &other)=delete
- Component & **operator=** (const Component &other)=delete
- **Component** (Component &&other)=delete
- Component & **operator=** (Component &&other)=delete
- int **id** () const
- std::string & **label** ()
- std::string **label** () const
- std::vector< Port * > & **inputs** ()
- std::vector< Port * > **inputs** () const
- std::vector< Port * > & **outputs** ()
- std::vector< Port * > **outputs** () const
- Port * **add_input** (int width=1, const std::string &label="")
- Port * **add_output** (int width=1, const std::string &label="")
- Port * **input_port** (int i)
- Port * **input_port** (int i) const
- Port * **output_port** (int i)
- Port * **output_port** (int i) const
- int **input_count** () const
- int **output_count** () const

Private Attributes

- PinType **type_**
- LogicVector **value_**

4.10.1 Member Function Documentation

4.10.1.1 evaluate()

```
void Pin::evaluate (
    Simulator & simulator) [inline], [virtual]
```

Implements Component.

4.10.1.2 print()

```
void Pin::print (
    std::ostream & cout) const [inline], [virtual]
```

Reimplemented from [Component](#).

4.10.1.3 reset()

```
void Pin::reset () [inline], [virtual]
```

Reimplemented from [Component](#).

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Logisimp/include/Pin.h

4.11 Port Class Reference

Public Member Functions

- **Port** (int id, int width, PortDirection direction, const std::string &label="")
- **Port** (const [Port](#) &other)=delete
- [Port](#) & **operator=** (const [Port](#) &other)=delete
- int & **id** ()
- int **id** () const
- int & **width** ()
- int **width** () const
- PortDirection & **direction** ()
- PortDirection **direction** () const
- [Component](#) *& **owner** ()
- [Component](#) * **owner** () const
- [Wire](#) *& **wire** ()
- [Wire](#) * **wire** () const
- std::string & **label** ()
- std::string **label** () const
- bool **connected** () const

Private Attributes

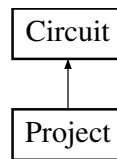
- int **id_**
- int **width_**
- PortDirection **direction_**
- [Component](#) * **owner_**
- [Wire](#) * **wire_**
- std::string **label_**

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Logisimp/include/Port.h

4.12 Project Class Reference

Inheritance diagram for Project:



Public Member Functions

- void **print_truth_table** ()
- [Pin](#) & **add_pin** (PinType type, int width=1)
- int **input_size** () const
- int **output_size** () const
- int **input_width** (int i) const
- int **output_width** (int i) const
- bool **initialize** ()
- void **set_at** (int i, const [LogicVector](#) &value)
- void **sim_run** ()

Public Member Functions inherited from [Circuit](#)

- int **comp_size** () const
- [Component](#) *& **component_at** (int i)
- [Component](#) * **component_at** (int i) const
- **Circuit** (const [Circuit](#) &other)=delete
- [Circuit](#) & **operator=** (const [Circuit](#) &other)=delete
- std::vector< [Component](#) * > & **components** ()
- std::vector< [Component](#) * > **components** () const
- std::vector< [Wire](#) * > & **wires** ()
- std::vector< [Wire](#) * > **wires** () const
- [AndGate](#) & **create_and_gate** (int input_count=2)
- [OrGate](#) & **create_or_gate** (int input_count=2)
- [NotGate](#) & **create_not_gate** ()
- [NandGate](#) & **create_nand_gate** (int input_count=2)
- [NorGate](#) & **create_nor_gate** (int input_count=2)
- [XorGate](#) & **create_xor_gate** (int input_count=2)
- [XnorGate](#) & **create_xnor_gate** (int input_count=2)
- [Pin](#) & **create_pin** (PinType type, int width=1)
- [Wire](#) & **create_wire** (int width=1, const std::string &label="")
- void **connect** ([Port](#) &port, [Wire](#) &wire)
- void **disconnect** ([Port](#) &port)
- void **remove_component** ([Component](#) &component)
- void **remove_wire** ([Wire](#) &wire)
- bool **initialize** ([Simulator](#) &simulator)
- bool **reset** ([Simulator](#) &simulator)

Public Attributes

- `std::vector< Pin * > inputs_`
- `std::vector< Pin * > outputs_`
- `Simulator * simulator`

The documentation for this class was generated from the following file:

- `/home/student/Desktop/Project/Logisimp/include/Test.h`

4.13 Simulator Class Reference

Public Member Functions

- `std::queue< Component * > & work_queue ()`
- `std::queue< Component * > work_queue () const`
- `std::unordered_set< Component * > & queued_components ()`
- `std::unordered_set< Component * > queued_components () const`
- `void enqueue (Component *component)`
- `void drive (Wire &wire, const LogicVector &value)`
- `bool run ()`
- `void clear ()`

Private Attributes

- `std::queue< Component * > work_queue_`
- `std::unordered_set< Component * > queued_components_`

The documentation for this class was generated from the following file:

- `/home/student/Desktop/Project/Logisimp/include/Simulator.h`

4.14 TestProject Class Reference

Public Member Functions

- `void simi_run ()`
- `int available_circuits ()`
- `int project_options (const string &name)`
- `void available_projects ()`
- `int toggle_inputs (string &)`
- `int show_outputs (const string &)`
- `void test_input_output ()`
- `void test_NOT ()`
- `void test_AND ()`
- `void test_OR ()`
- `void test_NAND ()`
- `void test_NOR ()`

- void **test_XOR** ()
- void **test_XNOR** ()
- void **test_BUFFER** ()
- void **test_TRI_STATE_BUFFER** ()
- void **test_HALF_ADDER** ()
- void **test_FULL_ADDER** ()
- void **test_4BIT_ADDER** ()
- void **test_MULTIPLEXER** ()
- void **test_4_1_MUX** ()
- void **test_8_1_MUX** ()
- void **test_16_1_MUX** ()
- void **test_DEMULTIPLEXER** ()
- void **test_DECODER** ()
- void **test_ENCODER** ()

Private Attributes

- unordered_map< string, [Project](#) * > **sim**

The documentation for this class was generated from the following files:

- /home/student/Desktop/Project/Logisimp/include/Test.h
- /home/student/Desktop/Project/Logisimp/src/Test.cpp

4.15 Wire Class Reference

Public Member Functions

- **Wire** (int id, int width=1, const std::string &label="")
- **Wire** (const [Wire](#) &other)=delete
- **Wire & operator=** (const [Wire](#) &other)=delete
- int & **id** ()
- int **id** () const
- std::string & **label** ()
- std::string **label** () const
- [LogicVector](#) & **value** ()
- [LogicVector](#) **value** () const
- [Port](#) *& **driver** ()
- [Port](#) * **driver** () const
- std::vector< [Port](#) * > & **listeners** ()
- std::vector< [Port](#) * > **listeners** () const
- int **width** () const

Private Attributes

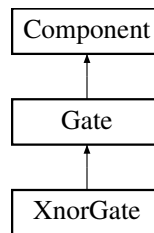
- int **id_**
- std::string **label_**
- [LogicVector](#) **value_**
- [Port](#) * **driver_**
- std::vector< [Port](#) * > **listeners_**

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Logisimp/include/Wire.h

4.16 XnorGate Class Reference

Inheritance diagram for XnorGate:



Public Member Functions

- **XnorGate** (int id, int input_count=2)
- void **evaluate** ([Simulator](#) &simulator)
- void **print** (std::ostream &cout) const

Public Member Functions inherited from [Gate](#)

- **Gate** (int id, int input_count)
- [Port](#) * **output** ()
- [Port](#) * **output** () const

Public Member Functions inherited from [Component](#)

- **Component** (int id, const std::string &label="")
- **Component** (const [Component](#) &other)=delete
- [Component](#) & **operator=** (const [Component](#) &other)=delete
- **Component** ([Component](#) &&other)=delete
- [Component](#) & **operator=** ([Component](#) &&other)=delete
- int **id** () const
- std::string & **label** ()
- std::string **label** () const
- std::vector< [Port](#) * > & **inputs** ()
- std::vector< [Port](#) * > **inputs** () const
- std::vector< [Port](#) * > & **outputs** ()
- std::vector< [Port](#) * > **outputs** () const
- virtual void **reset** ()
- [Port](#) * **add_input** (int width=1, const std::string &label="")
- [Port](#) * **add_output** (int width=1, const std::string &label="")
- [Port](#) * **input_port** (int i)
- [Port](#) * **input_port** (int i) const
- [Port](#) * **output_port** (int i)
- [Port](#) * **output_port** (int i) const
- int **input_count** () const
- int **output_count** () const

Additional Inherited Members

Protected Member Functions inherited from [Gate](#)

- `LogicValue` **input_value** (int index) const
- void **drive_output** (`LogicValue` value, [Simulator](#) &simulator)

4.16.1 Member Function Documentation

4.16.1.1 `evaluate()`

```
void XnorGate::evaluate (
    Simulator & simulator) [inline], [virtual]
```

Implements [Component](#).

4.16.1.2 `print()`

```
void XnorGate::print (
    std::ostream & cout) const [inline], [virtual]
```

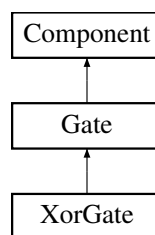
Reimplemented from [Gate](#).

The documentation for this class was generated from the following file:

- `/home/student/Desktop/Project/Logisimp/include/Gates.h`

4.17 XorGate Class Reference

Inheritance diagram for XorGate:



Public Member Functions

- **XorGate** (int id, int input_count=2)
- void **evaluate** ([Simulator](#) &simulator)
- void **print** (std::ostream &cout) const

Public Member Functions inherited from [Gate](#)

- **Gate** (int id, int input_count)
- [Port](#) * **output** ()
- [Port](#) * **output** () const

Public Member Functions inherited from [Component](#)

- **Component** (int id, const std::string &label="")
- **Component** (const [Component](#) &other)=delete
- [Component](#) & **operator=** (const [Component](#) &other)=delete
- **Component** ([Component](#) &&other)=delete
- [Component](#) & **operator=** ([Component](#) &&other)=delete
- int **id** () const
- std::string & **label** ()
- std::string **label** () const
- std::vector< [Port](#) * > & **inputs** ()
- std::vector< [Port](#) * > **inputs** () const
- std::vector< [Port](#) * > & **outputs** ()
- std::vector< [Port](#) * > **outputs** () const
- virtual void **reset** ()
- [Port](#) * **add_input** (int width=1, const std::string &label="")
- [Port](#) * **add_output** (int width=1, const std::string &label="")
- [Port](#) * **input_port** (int i)
- [Port](#) * **input_port** (int i) const
- [Port](#) * **output_port** (int i)
- [Port](#) * **output_port** (int i) const
- int **input_count** () const
- int **output_count** () const

Additional Inherited Members

Protected Member Functions inherited from [Gate](#)

- LogicValue **input_value** (int index) const
- void **drive_output** (LogicValue value, [Simulator](#) &simulator)

4.17.1 Member Function Documentation

4.17.1.1 evaluate()

```
void XorGate::evaluate (
    Simulator & simulator) [inline], [virtual]
```

Implements [Component](#).

4.17.1.2 print()

```
void XorGate::print (
    std::ostream & cout) const [inline], [virtual]
```

Reimplemented from [Gate](#).

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Logisimp/include/Gates.h

Chapter 5

File Documentation

5.1 Adder.h

```
00001 // Adder.h
00002
00003 #ifndef ADDER_H
00004 #define ADDER_H
00005
00006 #endif // Adder.h
```

5.2 Circuit.h

```
00001 // Circuit.h
00002
00003 #ifndef CIRCUIT_H
00004 #define CIRCUIT_H
00005
00006 #include "Component.h"
00007 #include "Gates.h"
00008 #include "Pin.h"
00009 #include "Port.h"
00010 #include "Simulator.h"
00011 #include "Wire.h"
00012
00013 #include <algorithm>
00014 #include <stdexcept>
00015 #include <string>
00016 #include <vector>
00017
00018 class Circuit
00019 {
00020 public:
00021     Circuit()
00022         : next_component_id_(0), next_wire_id_(0)
00023     {}
00024
00025     ~Circuit()
00026     {
00027         for (Component * component : components_)
00028         {
00029             delete component;
00030         }
00031
00032         for (Wire * wire : wires_)
00033         {
00034             delete wire;
00035         }
00036     }
00037     int comp_size() const
00038     {
00039         return components_.size();
00040     }
00041     Component *& component_at(int i)
00042     {
00043         return components_[i];
00044     }
00045 }
```

```

00045
00046     Component * component_at(int i) const
00047     {
00048         return components_[i];
00049     }
00050     Circuit(const Circuit & other) = delete;
00051     Circuit & operator=(const Circuit & other) = delete;
00052
00053     std::vector< Component * > & components() { return components_; }
00054     std::vector< Component * > components() const { return components_; }
00055
00056     std::vector< Wire * > & wires() { return wires_; }
00057     std::vector< Wire * > wires() const { return wires_; }
00058
00059     // Creators
00060     //-----
00061     AndGate & create_and_gate(int input_count = 2)
00062     {
00063         AndGate * component = new AndGate(next_component_id_, input_count);
00064
00065         store_component(component);
00066
00067         return *component;
00068     }
00069
00070     OrGate & create_or_gate(int input_count = 2)
00071     {
00072         OrGate * component = new OrGate(next_component_id_, input_count);
00073
00074         store_component(component);
00075
00076         return *component;
00077     }
00078
00079     NotGate & create_not_gate()
00080     {
00081         NotGate * component = new NotGate(next_component_id_);
00082
00083         store_component(component);
00084
00085         return *component;
00086     }
00087
00088     NandGate & create_nand_gate(int input_count = 2)
00089     {
00090         NandGate * component = new NandGate(next_component_id_, input_count);
00091
00092         store_component(component);
00093
00094         return *component;
00095     }
00096
00097     NorGate & create_nor_gate(int input_count = 2)
00098     {
00099         NorGate * component = new NorGate(next_component_id_, input_count);
00100
00101         store_component(component);
00102
00103         return *component;
00104     }
00105
00106     XorGate & create_xor_gate(int input_count = 2)
00107     {
00108         XorGate * component = new XorGate(next_component_id_, input_count);
00109
00110         store_component(component);
00111
00112         return *component;
00113     }
00114
00115     XnorGate & create_xnor_gate(int input_count = 2)
00116     {
00117         XnorGate * component = new XnorGate(next_component_id_, input_count);
00118
00119         store_component(component);
00120
00121         return *component;
00122     }
00123
00124     Pin & create_pin(PinType type, int width = 1)
00125     {
00126         Pin * component = new Pin(next_component_id_, type, width);
00127
00128         store_component(component);
00129         // cout << "TYPE: " << type << endl;
00130         // cout << "inie: " << component->input_count() << endl;
00131         // cout << "outie: " << component->output_count() << endl;

```

```

00132
00133         return *component;
00134     }
00135
00136     Wire & create_wire(int width = 1, const std::string & label = "")
00137     {
00138         Wire * wire = new Wire(next_wire_id_, width, label);
00139
00140         wires_.push_back(wire);
00141         next_wire_id_++;
00142
00143         return *wire;
00144     }
00145
00146     void connect(Port & port, Wire & wire)
00147     {
00148
00149         if (port.owner() == nullptr)
00150         {
00151             throw std::runtime_error("Port does not have an owner");
00152         }
00153
00154         if (!owns_component(*port.owner()))
00155         {
00156             throw std::runtime_error("Port owner does not belong to this circuit");
00157         }
00158
00159
00160         if (!owns_wire(wire))
00161         {
00162             throw std::runtime_error("Wire does not belong to this circuit");
00163         }
00164
00165         if (port.wire() != nullptr)
00166         {
00167             throw std::runtime_error("Port is already connected");
00168         }
00169
00170         if (port.width() != wire.width())
00171         {
00172             throw std::runtime_error("Port and wire widths do not match");
00173         }
00174
00175         if (port.direction() == OUTPUT)
00176         {
00177             if (wire.driver() != nullptr)
00178             {
00179                 throw std::runtime_error("Wire already has a driver");
00180             }
00181
00182             wire.driver() = &port;
00183         }
00184         else
00185         {
00186             cout << "got you" << endl;
00187             wire.listeners().push_back(&port);
00188         }
00189
00190         port.wire() = &wire;
00191     }
00192
00193     // removes references to the port
00194     // (used before the port is deleted when deleting a component)
00195     void disconnect(Port & port)
00196     {
00197         if (port.wire() == nullptr)
00198         {
00199             return;
00200         }
00201
00202         Wire * wire = port.wire();
00203
00204         if (port.direction() == OUTPUT)
00205         {
00206             if (wire->driver() == &port)
00207             {
00208                 wire->driver() = nullptr;
00209             }
00210         }
00211         else
00212         {
00213             std::vector< Port * >::iterator position = std::find(wire->listeners().begin(),
00214 wire->listeners().end(), &port);
00215
00216             if (position != wire->listeners().end())
00217             {

```

```

00218         wire->listeners().erase(position);
00219     }
00220 }
00221
00222     port.wire() = nullptr;
00223 }
00224
00225 void remove_component(Component & component)
00226 {
00227     std::vector< Component * >::iterator position = std::find(components_.begin(),
components_.end(), &component);
00228
00229     if (position == components_.end())
00230     {
00231         throw std::runtime_error("Component does not belong to this circuit");
00232     }
00233
00234     for (Port * port : component.inputs())
00235     {
00236         disconnect(*port);
00237     }
00238
00239     for (Port * port : component.outputs())
00240     {
00241         disconnect(*port);
00242     }
00243
00244     components_.erase(position);
00245
00246     delete &component;
00247 }
00248
00249 void remove_wire(Wire & wire)
00250 {
00251     std::vector< Wire * >::iterator position = std::find(wires_.begin(), wires_.end(), &wire);
00252
00253     if (position == wires_.end())
00254     {
00255         throw std::runtime_error("Wire does not belong to this circuit");
00256     }
00257
00258     if (wire.driver() != nullptr)
00259     {
00260         disconnect(*wire.driver());
00261     }
00262
00263     while (!wire.listeners().empty())
00264     {
00265         disconnect(*wire.listeners().back());
00266     }
00267
00268     wires_.erase(position);
00269
00270     delete &wire;
00271 }
00272
00273 bool initialize(Simulator & simulator)
00274 {
00275     simulator.clear();
00276
00277     for (Component * component : components_)
00278     {
00279         simulator.enqueue(component);
00280     }
00281
00282     return simulator.run();
00283 }
00284
00285 bool reset(Simulator & simulator)
00286 {
00287     simulator.clear();
00288
00289     for (Wire * wire : wires_)
00290     {
00291         wire->value() = LogicVector(wire->width(), UNKNOWN);
00292     }
00293
00294     for (Component * component : components_)
00295     {
00296         component->reset();
00297         simulator.enqueue(component);
00298     }
00299
00300     return simulator.run();
00301 }
00302
00303 private:

```

```

00304     void store_component(Component * component)
00305     {
00306         components_.push_back(component);
00307         next_component_id++;
00308     }
00309
00310     bool owns_component(const Component & component) const
00311     {
00312         return std::find(components_.begin(), components_.end(), &component) != components_.end();
00313     }
00314
00315     bool owns_wire(const Wire & wire) const
00316     {
00317         return std::find(wires_.begin(), wires_.end(), &wire) != wires_.end();
00318     }
00319
00320     std::vector< Component * > components_;
00321     std::vector< Wire * > wires_;
00322
00323     int next_component_id_;
00324     int next_wire_id_;
00325 };
00326
00327 #endif // CIRCUIT_H

```

5.3 Clock.h

```

00001 // Clock.h
00002
00003 #ifndef CLOCK_H
00004 #define CLOCK_H
00005
00006 #endif // Clock.h

```

5.4 Component.h

```

00001 // Component.h
00002
00003 #ifndef COMPONENT_H
00004 #define COMPONENT_H
00005
00006 #include "Port.h"
00007
00008 #include <string>
00009 #include <vector>
00010 #include <iostream>
00011
00012 class Simulator;
00013
00014 class Component
00015 {
00016 public:
00017     Component(int id, const std::string & label = "")
00018         : id_(id), label_(label)
00019     {}
00020
00021     // Don't allow copying.
00022     // Copying should be managed by daddy Circuit
00023     Component(const Component & other) = delete;
00024     Component & operator=(const Component & other) = delete;
00025
00026     Component(Component && other) = delete;
00027     Component & operator=(Component && other) = delete;
00028
00029     virtual ~Component()
00030     {
00031         clear_ports();
00032     }
00033
00034     int id() const { return id_; }
00035
00036     std::string & label() { return label_; }
00037     std::string label() const { return label_; }
00038
00039     std::vector< Port * > & inputs() { return inputs_; }
00040     std::vector< Port * > inputs() const { return inputs_; }
00041
00042     std::vector< Port * > & outputs() { return outputs_; }

```

```

00043     std::vector< Port * > outputs() const { return outputs_; }
00044
00045     virtual void evaluate(Simulator & simulator) = 0;
00046
00047     virtual void reset()
00048     {}
00049
00050     virtual void print(std::ostream & cout) const
00051     {
00052         cout << "Component("
00053             << "id=" << id_
00054             << ", label=\"" << label_ << "\""
00055             << ", inputs=" << inputs_.size()
00056             << ", outputs=" << outputs_.size()
00057             << ')';
00058     }
00059
00060 // protected:
00061 Port * add_input(int width = 1, const std::string & label = "")
00062 {
00063     int port_id = inputs_.size() + outputs_.size();
00064
00065     Port * port = new Port(port_id, width, INPUT, label);
00066     port->owner() = this;
00067
00068     inputs_.push_back(port);
00069
00070     return port;
00071 }
00072
00073 Port * add_output(int width = 1, const std::string & label = "")
00074 {
00075     int port_id = inputs_.size() + outputs_.size();
00076
00077     Port * port = new Port(port_id, width, OUTPUT, label);
00078     port->owner() = this;
00079
00080     outputs_.push_back(port);
00081
00082     return port;
00083 }
00084
00085 Port * input_port(int i)
00086 {
00087     return inputs_.at(i); // at is [] but does bounds-checking
00088     // (throws std::out_of_range error if out of bounds)
00089 }
00090
00091 Port * input_port(int i) const
00092 {
00093     return inputs_.at(i);
00094 }
00095
00096 Port * output_port(int i)
00097 {
00098     return outputs_.at(i);
00099 }
00100
00101 Port * output_port(int i) const
00102 {
00103     return outputs_.at(i);
00104 }
00105
00106 int input_count() const
00107 {
00108     return inputs_.size();
00109 }
00110
00111 int output_count() const
00112 {
00113     return outputs_.size();
00114 }
00115
00116 private:
00117 void clear_ports()
00118 {
00119     for (Port * port : inputs_)
00120     {
00121         delete port;
00122     }
00123
00124     for (Port * port : outputs_)
00125     {
00126         delete port;
00127     }
00128
00129     inputs_.clear();

```

```

00130         outputs_.clear();
00131     }
00132
00133     int id_;
00134     std::string label_;
00135     std::vector< Port * > inputs_;
00136     std::vector< Port * > outputs_;
00137 };
00138
00139 inline std::ostream & operator<<(std::ostream & cout, const Component & component)
00140 {
00141     component.print(cout);
00142
00143     return cout;
00144 }
00145
00146 #endif // Component.h

```

5.5 Gate.h

```

00001 // Gate.h
00002
00003 #ifndef GATE_H
00004 #define GATE_H
00005
00006 #include "Component.h"
00007 #include "LogicValue.h"
00008 #include "Simulator.h"
00009
00010 #include <stdexcept>
00011
00012 class Gate : public Component
00013 {
00014 public:
00015     Gate(int id, int input_count)
00016         : Component(id)
00017     {
00018         if (input_count <= 0)
00019         {
00020             throw std::invalid_argument("Gate input count must be positive");
00021         }
00022
00023         for (int i = 0; i < input_count; i++)
00024         {
00025             add_input(1);
00026         }
00027
00028         add_output(1, "Y");
00029     }
00030
00031     Port * output()
00032     {
00033         return output_port(0);
00034     }
00035
00036     Port * output() const
00037     {
00038         return output_port(0);
00039     }
00040
00041     void print(std::ostream & cout) const
00042     {
00043         cout << "Gate("
00044             << "id=" << id()
00045             << ", inputs=" << input_count()
00046             << ')';
00047     }
00048
00049 protected:
00050     LogicValue input_value(int index) const
00051     {
00052         Port * port = input_port(index);
00053
00054         if (port->wire() == nullptr)
00055         {
00056             return UNKNOWN;
00057         }
00058
00059         return port->wire()->value()[0];
00060     }
00061
00062     void drive_output(LogicValue value, Simulator & simulator)
00063     {

```

```

00064         Port * port = output_port(0);
00065
00066         if (port->wire() == nullptr)
00067         {
00068             return;
00069         }
00070
00071         LogicVector output_value(1);
00072
00073         output_value[0] = value;
00074
00075         simulator.drive(*port->wire(), output_value);
00076     }
00077 };
00078
00079 #endif

```

5.6 Gates.h

```

00001 // Gates.h
00002
00003 #ifndef GATES_H
00004 #define GATES_H
00005
00006 #include "Gate.h"
00007
00008 class AndGate : public Gate
00009 {
00010 public:
00011     AndGate(int id, int input_count = 2)
00012         : Gate(id, input_count)
00013     {}
00014
00015     void evaluate(Simulator & simulator)
00016     {
00017         LogicValue result = HIGH;
00018
00019         for (int i = 0; i < input_count(); ++i)
00020         {
00021             result = logic_and(result, input_value(i));
00022
00023             if (result == LOW)
00024             {
00025                 break;
00026             }
00027         }
00028
00029         drive_output(result, simulator);
00030     }
00031
00032     void print(std::ostream & cout) const
00033     {
00034         cout << "AndGate ("
00035              << "id=" << id()
00036              << ", inputs=" << input_count()
00037              << ')';
00038     }
00039 };
00040
00041 class OrGate : public Gate
00042 {
00043 public:
00044     OrGate(int id, int input_count = 2)
00045         : Gate(id, input_count)
00046     {}
00047
00048     void evaluate(Simulator & simulator)
00049     {
00050         LogicValue result = LOW;
00051
00052         for (int i = 0; i < input_count(); ++i)
00053         {
00054             result = logic_or(result, input_value(i));
00055
00056             if (result == HIGH)
00057             {
00058                 break;
00059             }
00060         }
00061
00062         drive_output(result, simulator);
00063     }
00064 }

```

```

00065     void print(std::ostream & cout) const
00066     {
00067         cout << "OrGate("
00068             << "id=" << id()
00069             << ", inputs=" << input_count()
00070             << ')';
00071     }
00072 };
00073
00074 class NotGate : public Gate
00075 {
00076 public:
00077     NotGate(int id)
00078         : Gate(id, 1)
00079     {}
00080
00081     void evaluate(Simulator & simulator)
00082     {
00083         drive_output(logic_not(input_value(0)), simulator);
00084     }
00085
00086     void print(std::ostream & cout) const
00087     {
00088         cout << "NotGate("
00089             << "id=" << id()
00090             << ')';
00091     }
00092 };
00093
00094 class NandGate : public Gate
00095 {
00096 public:
00097     NandGate(int id, int input_count = 2)
00098         : Gate(id, input_count)
00099     {}
00100
00101     void evaluate(Simulator & simulator)
00102     {
00103         LogicValue result = HIGH;
00104
00105         for (int i = 0; i < input_count(); i++)
00106         {
00107             // Note that NAND is not associative,
00108             // which is why it gets not-ed at the end
00109             // and not here
00110             result = logic_and(result, input_value(i));
00111
00112             if (result == LOW)
00113             {
00114                 break;
00115             }
00116         }
00117
00118         drive_output(logic_not(result), simulator);
00119     }
00120
00121     void print(std::ostream & cout) const
00122     {
00123         cout << "NandGate("
00124             << "id=" << id()
00125             << ", inputs=" << input_count()
00126             << ')';
00127     }
00128 };
00129
00130 class NorGate : public Gate
00131 {
00132 public:
00133     NorGate(int id, int input_count = 2)
00134         : Gate(id, input_count)
00135     {}
00136
00137     void evaluate(Simulator & simulator)
00138     {
00139         LogicValue result = LOW;
00140
00141         for (int i = 0; i < input_count(); i++)
00142         {
00143             result = logic_or(result, input_value(i));
00144
00145             if (result == HIGH)
00146             {
00147                 break;
00148             }
00149         }
00150
00151         drive_output(logic_not(result), simulator);

```

```

00152     }
00153
00154     void print(std::ostream & cout) const
00155     {
00156         cout << "NorGate("
00157             << "id=" << id()
00158             << ", inputs=" << input_count()
00159             << ')';
00160     }
00161 };
00162
00163 class XorGate : public Gate
00164 {
00165 public:
00166     XorGate(int id, int input_count = 2)
00167         : Gate(id, input_count)
00168     {}
00169
00170     void evaluate(Simulator & simulator)
00171     {
00172         LogicValue result = LOW;
00173
00174         for (int i = 0; i < input_count(); i++)
00175         {
00176             result = logic_xor(result, input_value(i));
00177
00178             if (result == UNKNOWN)
00179             {
00180                 break;
00181             }
00182         }
00183
00184         drive_output(result, simulator);
00185     }
00186
00187     void print(std::ostream & cout) const
00188     {
00189         cout << "XorGate("
00190             << "id=" << id()
00191             << ", inputs=" << input_count()
00192             << ')';
00193     }
00194 };
00195
00196 class XnorGate : public Gate
00197 {
00198 public:
00199     XnorGate(int id, int input_count = 2)
00200         : Gate(id, input_count)
00201     {}
00202
00203     void evaluate(Simulator & simulator)
00204     {
00205         LogicValue result = LOW;
00206
00207         for (int i = 0; i < input_count(); i++)
00208         {
00209             result = logic_xor(result, input_value(i));
00210
00211             if (result == UNKNOWN)
00212             {
00213                 break;
00214             }
00215         }
00216
00217         drive_output(logic_not(result), simulator);
00218     }
00219
00220     void print(std::ostream & cout) const
00221     {
00222         cout << "XnorGate("
00223             << "id=" << id()
00224             << ", inputs=" << input_count()
00225             << ')';
00226     }
00227 };
00228
00229 #endif // Gates.h

```

5.7 LogicValue.h

```

00001 // LogicValue.h
00002

```

```

00003 #ifndef LOGIC_VALUE_H
00004 #define LOGIC_VALUE_H
00005
00006 #include <iostream>
00007 #include <exception>
00008
00009 using namespace std;
00010
00011 enum LogicValue {
00012     LOW = 0,
00013     HIGH = 1,
00014     UNKNOWN = 2
00015 };
00016
00017 // Shortens LogicValue to LV (only within this file though)
00018 #define LV LogicValue
00019
00020 const LV AND_TABLE[3][3] = {
00021     //           b = LOW   b = HIGH   b = UNKNOWN
00022     /* a = LOW   */ { LOW,    LOW,    LOW    },
00023     /* a = HIGH  */ { LOW,    HIGH,   UNKNOWN },
00024     /* a = UNKNOWN */ { LOW,    UNKNOWN, UNKNOWN }
00025 };
00026
00027 const LV OR_TABLE[3][3] = {
00028     //           b = LOW   b = HIGH   b = UNKNOWN
00029     /* a = LOW   */ { LOW,    HIGH,   UNKNOWN },
00030     /* a = HIGH  */ { HIGH,   HIGH,   HIGH    },
00031     /* a = UNKNOWN */ { UNKNOWN, HIGH,   UNKNOWN }
00032 };
00033
00034 const LV NOT_TABLE[3] = {
00035     //           input
00036     /* LOW      */ HIGH,
00037     /* HIGH     */ LOW,
00038     /* UNKNOWN  */ UNKNOWN
00039 };
00040
00041 const LV XOR_TABLE[3][3] = {
00042     //           b = LOW   b = HIGH   b = UNKNOWN
00043     /* a = LOW   */ { LOW,    HIGH,   UNKNOWN },
00044     /* a = HIGH  */ { HIGH,   LOW,    UNKNOWN },
00045     /* a = UNKNOWN */ { UNKNOWN, UNKNOWN, UNKNOWN }
00046 };
00047
00048 const LV NAND_TABLE[3][3] = {
00049     //           b = LOW   b = HIGH   b = UNKNOWN
00050     /* a = LOW   */ { HIGH,   HIGH,   HIGH    },
00051     /* a = HIGH  */ { HIGH,   LOW,    UNKNOWN },
00052     /* a = UNKNOWN */ { HIGH,   UNKNOWN, UNKNOWN }
00053 };
00054
00055 const LV NOR_TABLE[3][3] = {
00056     //           b = LOW   b = HIGH   b = UNKNOWN
00057     /* a = LOW   */ { HIGH,   LOW,    UNKNOWN },
00058     /* a = HIGH  */ { LOW,    LOW,    LOW     },
00059     /* a = UNKNOWN */ { UNKNOWN, LOW,    UNKNOWN }
00060 };
00061
00062 const LV XNOR_TABLE[3][3] = {
00063     //           b = LOW   b = HIGH   b = UNKNOWN
00064     /* a = LOW   */ { HIGH,   LOW,    UNKNOWN },
00065     /* a = HIGH  */ { LOW,    HIGH,   UNKNOWN },
00066     /* a = UNKNOWN */ { UNKNOWN, UNKNOWN, UNKNOWN }
00067 };
00068
00069 inline LV logic_and(LV a, LV b)
00070 {
00071     return AND_TABLE[a][b];
00072 }
00073
00074 inline LV logic_or(LV a, LV b)
00075 {
00076     return OR_TABLE[a][b];
00077 }
00078
00079 inline LV logic_not(LV value)
00080 {
00081     return NOT_TABLE[value];
00082 }
00083
00084 inline LV logic_xor(LV a, LV b)
00085 {
00086     return XOR_TABLE[a][b];
00087 }
00088
00089 inline LV logic_nand(LV a, LV b)

```

```

00090 {
00091     return NAND_TABLE[a][b];
00092 }
00093
00094 inline LV logic_nor(LV a, LV b)
00095 {
00096     return NOR_TABLE[a][b];
00097 }
00098
00099 inline LV logic_xnor(LV a, LV b)
00100 {
00101     return XNOR_TABLE[a][b];
00102 }
00103
00104 // undo macro for LogicValue
00105 #undef LV
00106
00107 inline std::ostream & operator<<(std::ostream & cout, LogicValue value)
00108 {
00109     if (value == LOW)
00110     {
00111         cout << "0";
00112     }
00113     else if (value == HIGH)
00114     {
00115         cout << "1";
00116     }
00117     else
00118     {
00119         cout << "X";
00120     }
00121     return cout;
00122 }
00123
00124
00125 inline std::istream & operator>>(std::istream & cin, LogicValue & value)
00126 {
00127     int v;
00128     cin >> v;
00129
00130     if (v == 0)
00131         value = LOW;
00132     else if (v == 1)
00133         value = HIGH;
00134     else if (v == 2)
00135         value = UNKNOWN;
00136     else
00137         cout << "No value\n";
00138
00139     return cin;
00140 }
00141
00142 #endif // LOGIC_VALUE_H

```

5.8 LogicVector.h

```

00001 // LogicVector.h
00002
00003 #ifndef LOGIC_VECTOR_H
00004 #define LOGIC_VECTOR_H
00005
00006 #include "LogicValue.h"
00007
00008 #include <stdexcept>
00009 #include <vector>
00010 #include <iostream>
00011
00012 class LogicVector
00013 {
00014 public:
00015     LogicVector(int width = 1, LogicValue initial_value = UNKNOWN)
00016     {
00017         if (width <= 0)
00018         {
00019             throw std::invalid_argument("LogicVector width must be positive");
00020         }
00021
00022         bits_.resize(width, initial_value);
00023     }
00024
00025     std::vector< LogicValue > & bits() { return bits_; }
00026     std::vector< LogicValue > bits() const { return bits_; }
00027

```

```

00028     int width() const
00029     {
00030         return bits_.size();
00031     }
00032
00033     LogicValue operator[](int index) const
00034     {
00035         return bits_.at(index);
00036     }
00037
00038     LogicValue & operator[](int index)
00039     {
00040         return bits_.at(index);
00041     }
00042
00043     bool operator==(const LogicVector & other) const
00044     {
00045         return bits_ == other.bits_;
00046     }
00047
00048     bool operator!=(const LogicVector & other) const
00049     {
00050         return bits_ != other.bits_;
00051     }
00052
00053     bool fully_known() const
00054     {
00055         for (LogicValue bit : bits_)
00056         {
00057             if (bit == UNKNOWN)
00058             {
00059                 return false;
00060             }
00061         }
00062         return true;
00063     }
00064
00065 private:
00066     std::vector< LogicValue > bits_;
00067 };
00068
00069 inline std::ostream & operator<<(std::ostream & cout, const LogicVector & value)
00070 {
00071     cout << '[';
00072     for (int i = 0; i < value.width(); i++)
00073     {
00074         if (i != 0)
00075         {
00076             cout << ' ';
00077         }
00078         cout << value[i];
00079     }
00080     cout << ']';
00081     return cout;
00082 }
00083
00084 #endif // LogicVector.h

```

5.9 Mux.h

```

00001 // Mux.h
00002
00003 #ifndef MUX_H
00004 #define MUX_H
00005
00006 #endif // Mux.h

```

5.10 Pin.h

```

00001 // Pin.h
00002
00003 #ifndef PIN_H
00004 #define PIN_H

```

```

00005
00006 #include "Component.h"
00007 #include "Simulator.h"
00008
00009 #include <stdexcept>
00010
00011 enum PinType
00012 {
00013     INPUT_PIN,
00014     OUTPUT_PIN
00015 };
00016
00017 class Pin : public Component
00018 {
00019 public:
00020     Pin(int id, PinType type, int width = 1)
00021         : Component(id), type_(type), value_(width, type == INPUT_PIN ? LOW : UNKNOWN)
00022     {
00023         if (type == INPUT_PIN)
00024         {
00025             add_output(width, "Y");
00026         }
00027         else
00028         {
00029             add_input(width, "A");
00030         }
00031     }
00032
00033     PinType & type() { return type_; }
00034     PinType type() const { return type_; }
00035
00036     LogicVector & value() { return value_; }
00037     LogicVector value() const { return value_; }
00038
00039     void set(const LogicVector & value, Simulator & simulator)
00040     {
00041         if (type_ != INPUT_PIN)
00042         {
00043             throw std::logic_error("Cannot set an output pin");
00044         }
00045
00046         if (value.width() != value_.width())
00047         {
00048             throw std::invalid_argument("Pin value width does not match pin width");
00049         }
00050
00051         value_ = value;
00052
00053         evaluate(simulator);
00054     }
00055
00056     void toggle(Simulator & simulator)
00057     {
00058         if (type_ != INPUT_PIN)
00059         {
00060             throw std::logic_error("Cannot toggle an output pin");
00061         }
00062
00063         for (int i = 0; i < value_.width(); i++)
00064         {
00065             if (value_[i] == LOW)
00066             {
00067                 value_[i] = HIGH;
00068             }
00069             else if (value_[i] == HIGH)
00070             {
00071                 value_[i] = LOW;
00072             }
00073         }
00074
00075         evaluate(simulator);
00076     }
00077
00078     void evaluate(Simulator & simulator)
00079     {
00080         if (type_ == INPUT_PIN)
00081         {
00082             Port * port = output_port(0);
00083
00084             if (port->wire() != nullptr)
00085             {
00086                 simulator.drive(*port->wire(), value_);
00087             }
00088
00089             return;
00090         }
00091

```

```

00092         Port * port = input_port(0);
00093
00094         if (port->wire() == nullptr)
00095         {
00096             value_ = LogicVector(value_.width(), UNKNOWN);
00097             return;
00098         }
00099
00100         value_ = port->wire()->value();
00101     }
00102
00103     void reset()
00104     {
00105         if (type_ == INPUT_PIN)
00106         {
00107             value_ = LogicVector(value_.width(), LOW);
00108         }
00109         else
00110         {
00111             value_ = LogicVector(value_.width(), UNKNOWN);
00112         }
00113     }
00114
00115     void print(std::ostream & cout) const
00116     {
00117         cout << "Pin("
00118              << "id=" << id()
00119              << ", type=";
00120
00121         if (type_ == INPUT_PIN)
00122         {
00123             cout << "INPUT_PIN";
00124         }
00125         else
00126         {
00127             cout << "OUTPUT_PIN";
00128         }
00129
00130         cout << ", width=" << value_.width()
00131              << ", value=" << value_
00132              << ')';
00133     }
00134
00135 private:
00136     PinType type_;
00137     LogicVector value_;
00138 };
00139
00140 #endif // Pin.h

```

5.11 Port.h

```

00001 // Port.h
00002
00003 #ifndef PORT_H
00004 #define PORT_H
00005
00006 #include <string>
00007 #include <stdexcept>
00008 #include <iostream>
00009
00010 class Component;
00011 class Wire;
00012
00013 enum PortDirection
00014 {
00015     INPUT,
00016     OUTPUT
00017 };
00018
00019 class Port
00020 {
00021 public:
00022     Port(int id, int width, PortDirection direction, const std::string & label = "")
00023         : id_(id),
00024           width_(width),
00025           direction_(direction),
00026           owner_(nullptr),
00027           wire_(nullptr),
00028           label_(label)
00029     {
00030         if (width <= 0)
00031         {

```

```

00032         throw std::invalid_argument("Port width must be positive");
00033     }
00034 }
00035
00036 Port(const Port & other) = delete;
00037 Port & operator=(const Port & other) = delete;
00038
00039 int & id() { return id_; }
00040 int id() const { return id_; }
00041
00042 int & width() { return width_; }
00043 int width() const { return width_; }
00044
00045 PortDirection & direction() { return direction_; }
00046 PortDirection direction() const { return direction_; }
00047
00048 Component * & owner() { return owner_; }
00049 Component * owner() const { return owner_; }
00050
00051 Wire * & wire() { return wire_; }
00052 Wire * wire() const { return wire_; }
00053
00054 std::string & label() { return label_; }
00055 std::string label() const { return label_; }
00056
00057 bool connected() const
00058 {
00059     return wire_ != nullptr;
00060 }
00061
00062 private:
00063     int id_;
00064     int width_;
00065     PortDirection direction_;
00066     Component * owner_;
00067     Wire * wire_;
00068     std::string label_;
00069 };
00070
00071 inline std::ostream & operator<<(std::ostream & cout, const Port & port)
00072 {
00073     cout << "Port("
00074         << "id=" << port.id()
00075         << ", width=" << port.width()
00076         << ", direction=";
00077
00078     if (port.direction() == INPUT)
00079     {
00080         cout << "INPUT";
00081     }
00082     else
00083     {
00084         cout << "OUTPUT";
00085     }
00086
00087     cout << ", label=\"" << port.label() << "\""
00088         << ", connected=";
00089
00090     if (port.connected())
00091     {
00092         cout << "true";
00093     }
00094     else
00095     {
00096         cout << "false";
00097     }
00098
00099     cout << ')';
00100
00101     return cout;
00102 }
00103
00104 #endif // Port.h

```

5.12 RAM.h

```

00001 // RAM.h
00002
00003 #ifndef RAM_H
00004 #define RAM_H
00005
00006 #endif // RAM.h

```

5.13 Register.h

```
00001 // Register.h
00002
00003 #ifndef REGISTER_H
00004 #define REGISTER_H
00005
00006 #endif // Register.h
```

5.14 ROM.h

```
00001 // ROM.h
00002
00003 #ifndef ROM_H
00004 #define ROM_H
00005
00006 #endif // ROM.h
```

5.15 Simulator.h

```
00001 // Simulator.h
00002
00003 #ifndef SIMULATOR_H
00004 #define SIMULATOR_H
00005
00006 #include "Component.h"
00007 #include "LogicVector.h"
00008 #include "Wire.h"
00009
00010 #include <queue>
00011 #include <stdexcept>
00012 #include <unordered_set>
00013 #include <iostream>
00014
00015 class Simulator
00016 {
00017 public:
00018     Simulator()
00019     {}
00020
00021     std::queue< Component * > & work_queue() { return work_queue_; }
00022     std::queue< Component * > work_queue() const { return work_queue_; }
00023
00024     std::unordered_set< Component * > & queued_components() { return queued_components_; }
00025     std::unordered_set< Component * > queued_components() const { return queued_components_; }
00026
00027     void enqueue(Component * component)
00028     {
00029         if (component == nullptr)
00030         {
00031             return;
00032         }
00033
00034         if (queued_components_.find(component) != queued_components_.end())
00035         {
00036             return;
00037         }
00038
00039         queued_components_.insert(component);
00040         work_queue_.push(component);
00041     }
00042
00043     void drive(Wire & wire, const LogicVector & value)
00044     {
00045         if (wire.width() != value.width())
00046         {
00047             throw std::runtime_error("Cannot drive a wire with a value of a different width");
00048         }
00049
00050         // only enqueue if value changed
00051         if (wire.value() == value)
00052         {
00053             return;
00054         }
00055
00056         wire.value() = value;
00057     }
00058 }
```

```

00058         for (Port * listener : wire.listeners())
00059         {
00060             enqueue(listener->owner());
00061         }
00062     }
00063
00064     bool run()
00065     {
00066         const int MAX_EVALS = 100000;
00067         int evaluation_count = 0;
00068
00069         while (!work_queue_.empty())
00070         {
00071             // returns false if circuit oscillate
00072             // (need a better way to indicate this.
00073             // Currently the issue is evaluation count is global,
00074             // but it should be localized to each component)
00075             if (evaluation_count >= MAX_EVALS)
00076             {
00077                 clear();
00078                 return false;
00079                 // the issue with this is it aborts the whole circuit
00080                 // if one part is oscillating, but you can have
00081                 // two disconnected "circuits" in one circuit project.
00082                 // if one fails, we don't want that to make the other fail.
00083                 // Need a better way to do this. Probably by having a
00084                 // hashtable that maps Component * to evaluation_counts,
00085                 // then marking the output of components that exceed MAX_EVALS
00086                 // as UNKNOWN
00087             }
00088
00089             Component * component = work_queue_.front();
00090
00091             work_queue_.pop();
00092             queued_components_.erase(component);
00093
00094             evaluation_count++;
00095
00096             component->evaluate(*this);
00097         }
00098
00099         return true;
00100     }
00101
00102     void clear()
00103     {
00104         while (!work_queue_.empty())
00105         {
00106             work_queue_.pop();
00107         }
00108
00109         queued_components_.clear();
00110     }
00111
00112 private:
00113     std::queue< Component * > work_queue_;
00114     std::unordered_set< Component * > queued_components_;
00115 };
00116
00117 inline std::ostream & operator<<(std::ostream & cout, const Simulator & simulator)
00118 {
00119     std::queue< Component * > queue = simulator.work_queue();
00120
00121     cout << "Simulator("
00122           << "queued=" << queue.size()
00123           << ')';
00124
00125     if (!queue.empty())
00126     {
00127         cout << "\nQueue:";
00128
00129         while (!queue.empty())
00130         {
00131             cout << "\n  ";
00132
00133             if (queue.front() == nullptr)
00134             {
00135                 cout << "nullptr";
00136             }
00137             else
00138             {
00139                 cout << *queue.front();
00140             }
00141
00142             queue.pop();
00143         }
00144     }

```

```

00145
00146     return cout;
00147 }
00148
00149 #endif // Simulator.h

```

5.16 Test.h

```

00001 #ifndef TEST_H
00002 #define TEST_H
00003
00004 // Simulator:
00005 #include <unordered_map>
00006 #include "LogicValue.h"
00007 #include "LogicVector.h"
00008 #include "Port.h"
00009 #include "Component.h"
00010 #include "Wire.h"
00011 #include "Simulator.h"
00012 #include "Circuit.h"
00013 #include "Gate.h"
00014 #include "Gates.h"
00015 #include "Pin.h"
00016 // #include "Clock.h"
00017 // #include "Mux.h"
00018 // #include "Adder.h"
00019 // #include "Register.h"
00020 // #include "RAM.h"
00021 // #include "ROM.h"
00022
00023 class Project: public Circuit
00024 {
00025 public:
00026     Project()
00027         : simulator(new Simulator)
00028     {}
00029     ~Project()
00030     {
00031         delete simulator;
00032     }
00033     void print_truth_table(); // TODO
00034     Pin & add_pin(PinType type, int width = 1)
00035     {
00036         Pin & p = create_pin(type, width);
00037         if (type == INPUT_PIN)
00038             inputs_.push_back(&p);
00039         else
00040             outputs_.push_back(&p);
00041
00042         return p;
00043     }
00044     int input_size() const
00045     {
00046         return inputs_.size();
00047     }
00048     int output_size() const
00049     {
00050         return outputs_.size();
00051     }
00052     int input_width(int i) const
00053     {
00054         // cout << i << ' ' << inputs_[i] << endl;
00055         // Component * c = component_at(inputs_[i]);
00056         // cout << "get component: " << c << endl;
00057         // cout << "size: " << c->input_count() << endl;
00058         // cout << "input port at " << c->input_port(0) << endl;
00059         return (outputs_[i]->input_port(0))->width();
00060     }
00061     int output_width(int i) const
00062     {
00063         return (inputs_[i]->output_port(0))->width();
00064     }
00065     bool initialize()
00066     {
00067         return Circuit::initialize(*simulator);
00068     }
00069     void set_at(int i, const LogicVector & value)

```

```

00076     {
00077         Pin & p = *(inputs_[i]);
00078         p.set(value, *simulator);
00079     }
00080
00081     void sim_run()
00082     {
00083         simulator->run();
00084     }
00085
00086     std::vector<Pin *> inputs_;
00087     std::vector<Pin *> outputs_;
00088     Simulator * simulator;
00089 };
00090
00091 class TestProject
00092 {
00093 public:
00094     TestProject() //(const string & s) for directory
00095     {
00096         sim["main"] = new Project;
00097         test_input_output();
00098
00099         test_NOT();
00100         test_AND();
00101         test_OR();
00102         test_NAND();
00103         // test_NOR();
00104         // test_XOR();
00105         // test_XNOR();
00106
00107         // test_BUFFER();
00108         // test_TRI_STATE_BUFFER();
00109
00110         // test_HALF_ADDER();
00111         // test_FULL_ADDER();
00112         // test_4BIT_ADDER();
00113
00114         // test_MULTIPLEXER();
00115         // test_4_1_MUX();
00116         // test_8_1_MUX();
00117         // test_16_1_MUX();
00118         // test_DEMULTIPLEXER();
00119         // test_DECODER();
00120         // test_ENCODER();
00121     }
00122     ~TestProject()
00123     {
00124         for (auto p : sim)
00125         {
00126             delete p.second;
00127         }
00128     }
00129
00130     void simi_run();
00131     int available_circuits();
00132     int project_options(const string & name)
00133     {
00134         Project & pr = *sim[name];
00135         int outputs;
00136         cin >> outputs;
00137
00138         return outputs;
00139     }
00140     void available_projects();
00141     int toggle_inputs(string &);
00142     int show_outputs(const string &);
00143
00144     void test_input_output();
00145     void test_NOT();
00146     void test_AND();
00147     void test_OR();
00148     void test_NAND();
00149     void test_NOR();
00150     void test_XOR();
00151     void test_XNOR();
00152
00153     void test_BUFFER();
00154     void test_TRI_STATE_BUFFER();
00155
00156     void test_HALF_ADDER();
00157     void test_FULL_ADDER();
00158     void test_4BIT_ADDER();
00159
00160     void test_MULTIPLEXER();
00161     void test_4_1_MUX();
00162     void test_8_1_MUX();

```

```

00163     void test_16_1_MUX();
00164     void test_DEMULTIPLEXER();
00165     void test_DECODER();
00166     void test_ENCODER();
00167 private:
00168     unordered_map<string, Project *> sim;
00169 };
00170 #endif

```

5.17 Wire.h

```

00001 // Wire.h
00002
00003 #ifndef WIRE_H
00004 #define WIRE_H
00005
00006 #include "LogicVector.h"
00007 #include "Port.h"
00008
00009 #include <string>
00010 #include <vector>
00011 #include <iostream>
00012
00013 class Wire
00014 {
00015 public:
00016     Wire(int id, int width = 1, const std::string & label = "")
00017         : id_(id), label_(label), value_(width, UNKNOWN), driver_(nullptr)
00018     { }
00019
00020     Wire(const Wire & other) = delete;
00021     Wire & operator=(const Wire & other) = delete;
00022
00023     int & id() { return id_; }
00024     int id() const { return id_; }
00025
00026     std::string & label() { return label_; }
00027     std::string label() const { return label_; }
00028
00029     LogicVector & value() { return value_; }
00030     LogicVector value() const { return value_; }
00031
00032     Port * & driver() { return driver_; }
00033     Port * driver() const { return driver_; }
00034
00035     std::vector< Port * > & listeners() { return listeners_; }
00036     std::vector< Port * > listeners() const { return listeners_; }
00037
00038     int width() const
00039     {
00040         return value_.width();
00041     }
00042
00043 private:
00044     int id_;
00045     std::string label_;
00046     LogicVector value_;
00047     Port * driver_;
00048     std::vector< Port * > listeners_;
00049 };
00050
00051 inline std::ostream & operator<<(std::ostream & cout, const Wire & wire)
00052 {
00053     cout << "Wire("
00054         << "id=" << wire.id()
00055         << ", width=" << wire.width()
00056         << ", value=" << wire.value()
00057         << ", label=\"" << wire.label() << "\""
00058         << ", driver=";
00059
00060     if (wire.driver() == nullptr)
00061     {
00062         cout << "none";
00063     }
00064     else
00065     {
00066         cout << "port " << wire.driver()->id();
00067     }
00068
00069     cout << ", listeners=" << wire.listeners().size()
00070         << ')';
00071
00072     return cout;

```

```
00073 }  
00074  
00075 #endif // Wire.h
```

Index

/home/student/Desktop/Project/Logisimp/include/Adder.h, 27
XorGate, 26

/home/student/Desktop/Project/Logisimp/include/Circuit.h, 27
Gate, 10
print, 11

/home/student/Desktop/Project/Logisimp/include/Clock.h, 31
LogicVector, 12

/home/student/Desktop/Project/Logisimp/include/Component.h, 31
NandGate, 12
evaluate, 13

/home/student/Desktop/Project/Logisimp/include/Gate.h, 33
print, 13

/home/student/Desktop/Project/Logisimp/include/Gates.h, 34
NorGate, 14
evaluate, 15
print, 15

/home/student/Desktop/Project/Logisimp/include/LogicValue.h, 36
NotGate, 15
evaluate, 16

/home/student/Desktop/Project/Logisimp/include/LogicVector.h, 38
print, 16

/home/student/Desktop/Project/Logisimp/include/Mux.h, 39
OrGate, 17
evaluate, 18
print, 18

/home/student/Desktop/Project/Logisimp/include/Pin.h, 39
Pin, 18
evaluate, 19
print, 19
reset, 20

/home/student/Desktop/Project/Logisimp/include/Port.h, 41
Port, 20
print

/home/student/Desktop/Project/Logisimp/include/RAM.h, 42

/home/student/Desktop/Project/Logisimp/include/ROM.h, 43

/home/student/Desktop/Project/Logisimp/include/Register.h, 43
AndGate, 8
Gate, 11

/home/student/Desktop/Project/Logisimp/include/Simulator.h, 43
NandGate, 13
NorGate, 15
NotGate, 16
OrGate, 18
Pin, 19
XnorGate, 25
XorGate, 26

/home/student/Desktop/Project/Logisimp/include/Test.h, 45

/home/student/Desktop/Project/Logisimp/include/Wire.h, 47

AndGate, 7
evaluate, 8
print, 8

Circuit, 8

Component, 9

evaluate
AndGate, 8
NandGate, 13
NorGate, 15
NotGate, 16
OrGate, 18
Pin, 19
XnorGate, 25

Project, 21
reset
Pin, 20

Simulator, 22

TestProject, 22

Wire, 23

XnorGate, 24
evaluate, 25
print, 25

XorGate, 25
evaluate, 26
print, 26